NHẬT KÝ LÀM VIỆC
Tên đề tài: Xây dựng hệ thống đặt vé xem phim trực tuyến (Cinema Booking System) Công nghệ: Golang, PostgreSQL, Redis, WebSocket, JWT, VNPay Thời gian thực hiện:

17/01/2026 – 11/04/2026
Các thành viên:

- Phạm Thiên Phú *(Nhóm trưởng)*
- Nguyễn Quốc Tuấn
- Đào Thanh Tú
- Phạm Thanh Sự

Tuần 1: 17/01 – 24/01
Công việc thực hiện trong tuần:

- Thống nhất đề tài: Hệ thống đặt vé xem phim trực tuyến sử dụng ngôn ngữ Golang.
- Phân tích yêu cầu nghiệp vụ: Xác định các actor (Khách hàng, Quản trị viên), các use case chính và phụ.
- Nghiên cứu và so sánh công nghệ: Chọn Chi Router (thay vì Gin) vì lightweight; chọn pgx/v5 (thay vì GORM) vì hiệu suất cao với PostgreSQL.
- Lựa chọn kiến trúc: Clean Architecture với 4 tầng (Domain → Repository → Service → Handler).
- Phân công công việc lần đầu dựa trên năng lực và sở thích từng thành viên.

Kết quả buổi họp nhóm 1 (22/01/2026):

- Nội dung: Thống nhất quy trình làm việc, chọn đề tài, kiến trúc hệ thống.
- Kết quả đạt được:
    - Chọn đề tài Cinema Booking System với Golang.
    - Thống nhất Clean Architecture làm kiến trúc nền tảng. Mỗi layer chỉ phụ thuộc vào layer bên trong, không bao giờ ngược lại – đảm bảo khả năng test và thay thế từng phần độc lập.
    - Phân công: Phú (Nhóm trưởng + Race Condition + Admin), Tuấn (Auth + Payment), Tú (Catalog + Showtime), Sự (Frontend + WebSocket Client).
    - Thống nhất dùng GitHub với quy tắc Conventional Commits: feat:, fix:, refactor:.

Mục tiêu tuần sau:
Hoàn thành thiết kế Database (ERD) và danh sách API endpoints.

Tuần 2: 25/01 – 31/01
Công việc thực hiện trong tuần:

- Thiết kế Database (ERD): Xây dựng sơ đồ quan hệ thực thể với 9 bảng chính: users, movies, cinemas, screening_rooms, seats, showtimes, bookings, b ooking_seats, payments. Xác định khóa ngoại, ràng buộc và index cần thiết.
- Thiết kế API RESTful: Lập danh sách đầy đủ các endpoint theo nhóm chức năng (Auth, Movies, Cinemas, Showtimes, Bookings, Payments, Admin).
- Viết Migration SQL: Tạo script SQL khởi tạo toàn bộ schema database, bao gồm index tối ưu cho các truy vấn thường gặp.
- Setup môi trường phát triển: Docker Compose cho PostgreSQL và Redis đồng nhất trên máy tất cả thành viên. Khởi tạo module Go: go mod init booking_cinema_golang.

Kết quả buổi họp nhóm 1 (27/01/2026):

- Nội dung: Trình bày ERD, thiết kế API, thống nhất cấu trúc thư mục dự án.
- Kết quả đạt được:
    - ERD được toàn nhóm duyệt, xác định đầy đủ các quan hệ giữa bảng.
    - Danh sách API được phê duyệt. Thống nhất format response JSON chuẩn: { "data": ..., "error": ..., "pagination": ... }.
    - Cấu trúc thư mục cmd/api, internal/domain, internal/repository, internal/service, inter nal/handler, internal/middleware, internal/infrastructure được thiết lập.

Kết quả buổi họp nhóm 2 (29/01/2026):

- Nội dung: Giải đáp thắc mắc về ERD, thống nhất convention đặt tên field trong code Go (camelCase trong code, snake_case trong JSON/DB).

Mục tiêu tuần sau:
Hoàn thành Domain Models, cấu hình kết nối Database và Redis.

Tuần 3: 01/02 – 07/02
Công việc thực hiện trong tuần:

- Domain Models (Phú điều phối): Định nghĩa toàn bộ struct trong internal/domain/models.go: User, Cinema, Movie, ScreeningRoom, Seat, Showtime, Booking, Payment. Áp dụng struct tags JSON (json:"-" cho PasswordHash, json:"omitempty" cho optional fields) và Pointer Semantics (*string, *time.Time) cho nullable database columns.
- Repository Interfaces: Khai báo interface cho từng Repository với đầy đủ method signatures. Thống nhất pattern trả về (result, error) thay vì chỉ error.
- Infrastructure Layer: Module kết nối PostgreSQL qua jackc/pgx/v5 với connection pool (pgxpool). Module Redis client (go-redis/v9). Cấu hình từ environment variables.

- Structured Logging: Tích hợp uber-go/zap thay thế log standard library – hỗ trợ JSON format và log level.
- Sentinel Errors: Tập trung toàn bộ custom error variables vào internal/repository/errors.go để đảm bảo errors.Is() hoạt động đúng xuyên suốt toàn hệ thống.

Kết quả buổi họp nhóm 1 (04/02/2026):

- Nội dung: Review Domain Models, Repository Interfaces, thảo luận về error handling convention.
- Kết quả đạt được:
    - Domain models hoàn chỉnh với đầy đủ struct tags, pointer semantics.
    - Thống nhất dùng Sentinel Errors tập trung trong một file để tránh duplicate error với cùng message.
    - Module kết nối PostgreSQL và Redis hoạt động thành công trên máy tất cả thành viên.

Kết quả buổi họp nhóm 2 (06/02/2026):

- Nội dung: Giải đáp vướng mắc về thiết kế Repository Interface. Thống nhất: Repository chỉ làm database access, không chứa business logic.

Mục tiêu tuần sau:
Auth Module (JWT, Middleware), Catalog Module (Movies, Cinemas, Showtimes).

Tuần 4: 08/02 – 14/02

Công việc thực hiện trong tuần:

- Auth Module (Tuấn): Triển khai đăng ký tài khoản, đăng nhập, xác thực OTP qua email. Sử dụng bcrypt để hash mật khẩu trước khi lưu vào DB.
- JWT Utils (Tuấn): Hàm GenerateToken tạo JWT với Claims (UserID, Email, Username, ExpiresAt) ký bằng HMAC-SHA256.
  Hàm ParseToken giải mã và xác thực chữ ký, phát hiện token giả mạo hoặc hết hạn.
- Auth Middleware (Phú): Triển khai AuthMiddleware dạng Closure – capture jwtSecret từ config, trích xuất token từ header Authorization: Bearer <token>, gọi ParseToken, inject Claims vào context.Context bằng context.WithValue. Hàm GetClaims(ctx) cho Handler downstream dùng để lấy UserID.
- Catalog Module (Tú): Triển khai đầy đủ Repository và Service cho Movies, Cinemas, ScreeningRooms, Seats, Showtimes. API endpoint GET /api/v1/showtimes/movie/{id} lọc suất chiếu theo phim và ngày.
- Dependency Injection (Phú): Xây dựng cmd/api/main.go – wire toàn bộ dependencies theo thứ tự: Infrastructure → Repository → Service → Handler. Cấu hình Chi Router với middleware chain: RequestID, RealIP, Logger, Recoverer, Timeout.

Kết quả buổi họp nhóm 1 (17/02/2026):

- Nội dung: Demo Auth API (register, login, verify-otp), review Catalog API.
- Kết quả đạt được:
    - POST /api/v1/register, POST /api/v1/login, POST /api/v1/verify-otp hoạt động đúng.
    - JWT được tạo đúng format, Middleware xác thực thành công, Claims được inject vào Context.
    - Catalog endpoints (GET /movies, GET /cinemas, GET /showtimes) hoạt động với dữ liệu seed.

Kết quả buổi họp nhóm 2 (19/02/2026):

- Nội dung: Fix bug: OTP expiry không được kiểm tra đúng khi verify. Token refresh chưa được implement (đưa vào backlog, không thuộc phạm vi MVP).

Mục tiêu tuần sau:
WebSocket Hub, Redis Seat Lock (Layer 1 Race Condition), Booking Repository.

Tuần 5: 01/03 – 07/03
Công việc thực hiện trong tuần:

- WebSocket Hub (Sự + Phú): Triển khai ws_handler.go sử dụng Hub Pattern với Channels để broadcast trạng thái ghế real-time. Hub chạy trên một goroutine riêng với vòng lặp select vô hạn. Mỗi WebSocket connection có 2 goroutines: ReadPump (nhận từ client) và WritePump (gửi từ buffered channel). Khi một goroutine lag, Hub tự ngắt kết nối client đó qua default case để không block các client khác.
- Redis Seat Lock – Layer 1 (Phú): Triển khai LockSeat (SetNX với TTL), UnlockSeat (DEL key), IsSeatLocked, GetLockedSeats trong internal/infrastructure/redis/redis.go. Key format: seat_lock:{showtimeID}:{seatID}. Đây là lớp bảo vệ đầu tiên – nhanh (~1ms), distributed, tự giải phóng khi TTL hết.
- Booking Repository – Layer 2 (Phú): Triển khai PostgresBookingRepository.Create() với:
    - PostgreSQL Transaction (BEGIN/COMMIT/ROLLBACK).
    - SELECT ... FOR UPDATE NOWAIT (Pessimistic Lock) khóa hàng ghế, ORDER BY id để tránh Deadlock.
    - Double Check query: COUNT(*) kiểm tra booking confirmed hoặc pending < 5 phút. Nếu count > 0 → rollback và trả ErrSeatAlreadyTaken.
    - defer tx.Rollback(ctx) đảm bảo cleanup tự động dù panic hay early return.

- Booking Service (Phú): Kết hợp cả hai lớp lock – Redis Layer 1 cho tốc độ, DB Layer 2 làm authoritative check. Broadcast WebSocket sau mỗi thay đổi trạng thái ghế.
- Frontend cơ bản (Sự): Trang chủ, danh sách phim, trang chọn ghế với bản đồ ghế động kết nối WebSocket.

Kết quả buổi họp nhóm 1 (24/02/2026):

- Nội dung: Demo WebSocket real-time, kiểm tra Race Condition với 2 tab trình duyệt.
- Kết quả đạt được:
    - Khi User A chọn ghế A5, ghế đổi màu vàng trên màn hình User B ngay lập tức (< 100ms).
    - Demo Race Condition: 2 tab cùng nhấn ghế A5 → chỉ 1 thành công, tab còn lại nhận thông báo "Ghế đang bị giữ".
    - Transaction + FOR UPDATE NOWAIT hoạt động đúng: Không có double booking nào xảy ra.

Kết quả buổi họp nhóm 2 (26/02/2026):

- Nội dung: Fix bug: Redis lock chưa được giải phóng sau khi booking thành công. Thêm defer s.redis.UnlockSeat(ctx, ...) trong Service.

Mục tiêu tuần sau:
Payment Module (VNPay), Admin Module.

Tuần 6: 08/03 – 14/03
Công việc thực hiện trong tuần:

- Payment – Strategy Pattern (Tuấn + Phú): Định nghĩa interface PaymentProvider với 4 method (GetName, IsActive, CreatePayment, HandleWebhook). Triển khai VNPayProvider (HMAC-SHA512, sort params theo alphabet), MockProvider (simulate cash/MoMo cho dev/test). Service chọn provider tại runtime qua map[string]PaymentProvider.
- Idempotency (Phú): Triển khai checkIdempotency và cache kết quả vào Redis với TTL 24h. Acquire Distributed Lock trên Idempotency-Key trước khi xử lý để phòng 2 request cùng key đến đồng thời.
- SingleFlight (Phú): Áp dụng singleflight.Group.Do() cho GetPaymentMethods() – chỉ 1 goroutine thực sự query DB dù 100 goroutine gọi đồng thời.
- Webhook Async (Tuấn): Fire-and-Forget pattern: Goroutine xử lý VNPay webhook trong nền, trả 200 OK ngay. Bắt buộc dùng context.Background() thay vì r.Context() để goroutine không bị cancel khi HTTP response đã gửi.

- Admin Module (Phú): CRUD đầy đủ cho Movies, Cinemas, Rooms, Showtimes. API thống kê /admin/stats dùng SQL Aggregation (tổng vé, doanh thu, người dùng). Middleware AdminOnlyMiddleware kiểm tra JWT Claims role = "admin".
- Seat Status by Type (Phú): Đổi kiểu trả về của GetTakenSeatIDsForShowtime từ []string sang map[string]string (SeatID → "pending"/"confirmed") để Frontend phân biệt màu ghế.
- Seed Data (Tú): Viết seed.sql với dữ liệu thực tế – 10 phim, 3 rạp, nhiều suất chiếu theo ngày, tài khoản admin mặc định.

Kết quả buổi họp nhóm 1 (03/03/2026):

- Nội dung: Demo luồng thanh toán full end-to-end, review Admin Dashboard.
- Kết quả đạt được:
    - Chọn VNPay → Redirect sang trang VNPay → Callback → Booking confirmed → Real-time WebSocket update.
    - Admin Dashboard hiển thị số liệu thực từ DB (không hardcode).
    - Màu ghế phân biệt: Trống (xanh lá), Holding pending (vàng), Đã bán confirmed (đỏ/đen).

Kết quả buổi họp nhóm 2 (05/03/2026):

- Nội dung: Fix bug Idempotency: Khi 2 request cùng key đến đồng thời, cả 2 đều cache miss và xử lý 2 lần. Giải pháp: SETNX lock trước khi kiểm tra cache lần 2.

Mục tiêu tuần sau:
QA toàn diện, fix bug, Past Showtime Prevention, Countdown Timer.

Tuần 7: 15/03 – 21/03
Công việc thực hiện trong tuần:

- QA – Phát hiện và fix bug (Phú điều phối):

Bug #	Mức độ	Mô tả	Nguyên nhân	Giải pháp	Trạng thái
#001	Cao	Ghế hiển thị trống sau F5 trang thanh toán	Frontend chỉ lấy seat status từ Redis, không merge với DB pending	Merge Redis lock + DB pending booking trong API GET /seats/showtime/{id}	Fixed
#002	Trung bình	Đồng hồ đếm ngược không hiển thị hoặc về 0 ngay	API không trả về created_at của booking	Thêm RETURNING id::text, created_at trong SQL INSERT, trả về Frontend	Fixed

#003	Trung bình	Mất quyền thanh toán khi chuyển tab	Không có đường dẫn "Resume Payment"	Thêm nút "Thanh toán ngay" cho booking pending còn hạn trong "Vé của tôi"	Fixed
#004	Thấp	Vé hết hạn vẫn hiển thị trong "Vé của tôi"	ListByUserID không filter booking pending quá 5 phút	Thêm WHERE (status != 'pending' OR created_at > NOW() - INTERVAL '5 minutes')	Fixed
#005	Cao	Ghế pending vẫn có thể đặt được	Thiếu Double Check trong DB Transaction	Thêm COUNT(*) query kiểm tra booking active trước khi INSERT	Fixed

- Past Showtime Validation (Phú): Thêm kiểm tra startTime.Before(time.Now()) → ErrShowtimeExpired trong Repository. Frontend greyed-out và disable các suất chiếu đã qua.
- Countdown Timer (Sự): JavaScript tính thời gian còn lại dựa trên booking.created_at từ server (không dùng Date.now() của client để tránh clock skew giữa máy client và server). Auto redirect khi đồng hồ về 0.
- Date Calendar Filter (Sự): Bộ lọc ngày trên trang danh sách phim, hiển thị suất chiếu theo ngày được chọn.
- Graceful Shutdown (Phú): OS Signal handling (SIGINT, SIGTERM) với 15-second timeout context. defer cleanup theo thứ tự LIFO: logger.Sync() → rdb.Close() → db.Close().

Kết quả buổi họp nhóm 1 (17/03/2026):

- Nội dung: Review toàn bộ bug fix, test lại tất cả luồng nghiệp vụ chính sau fix.
- Kết quả đạt được:
  - Tất cả 5 bug đã fix và verify pass.
  - Countdown timer hoạt động đúng, không bị reset khi refresh trang.
  - Suất chiếu đã qua không thể đặt từ cả Frontend lẫn API.

Kết quả buổi họp nhóm 2 (19/03/2026):

- Nội dung: Kiểm thử bảo mật cơ bản lần 1. Phát hiện: Không có rate limiting trên Auth endpoints → Đưa vào danh sách cải thiện (không thuộc MVP). SQL Injection qua parameterized query → An toàn.

Mục tiêu tuần sau:
Integration Testing, hoàn thiện Frontend, viết tài liệu kỹ thuật.

Tuần 8: 22/03 – 28/03

Công việc thực hiện trong tuần:

- Integration Testing – Race Condition (Phú): Viết test với 10 goroutines đồng thời gọi CreateBooking cho cùng ghế. Chạy 10 lần, tất cả đều cho đúng 1

booking thành công. Xác nhận giải pháp Hybrid Lock hoạt động đúng trong mọi trường hợp.

- Kiểm thử bảo mật (Phú + Tuấn):

Kịch bản kiểm thử	Kết quả
API bảo vệ không có token	401 Unauthorized
JWT giả mạo (secret sai)	401 Unauthorized
JWT hết hạn	401 Unauthorized
Dùng JWT User A xem vé User B	Chỉ trả dữ liệu User A
Customer gọi Admin API	403 Forbidden
SQL Injection qua tham số	Parameterized query – An toàn
Đặt vé với booking_id người khác	Không tìm thấy dữ liệu

- Hoàn thiện Frontend (Sự): Responsive design cho mobile/tablet, UX loading states, thông báo lỗi thân thiện với người dùng, dark mode cho trang Admin.
- Viết báo cáo kỹ thuật (Cả nhóm):
    - Phú: Chương Race Condition (lý thuyết, use case, code), chương Admin, tổng quan kiến trúc.
    - Tuấn: Chương Auth JWT, chương Payment.
    - Tú: Chương Catalog, Showtime, Database design.
    - Sự: Chương Frontend, WebSocket real-time, hướng dẫn cài đặt.

Kết quả buổi họp nhóm 1 (24/03/2026):

- Nội dung: Review kết quả Integration Test, báo cáo tiến độ viết tài liệu.
- Kết quả đạt được:
    - Race Condition Integration Test pass 10/10 lần chạy.
    - Tất cả kịch bản bảo mật đều pass.
    - Phần báo cáo kỹ thuật đã hoàn thành khoảng 60%.

Kết quả buổi họp nhóm 2 (26/03/2026):

- Nội dung: Review draft báo cáo của từng thành viên, góp ý chỉnh sửa. Xác nhận danh sách tính năng "Done" cho demo.

Mục tiêu tuần sau:
Hoàn thiện báo cáo, chuẩn bị dữ liệu demo, viết Demo Script.

Tuần 9: 29/03 – 04/04

Công việc thực hiện trong tuần:

- Hoàn thiện báo cáo (Cả nhóm): Mỗi thành viên hoàn chỉnh phần của mình, review chéo để đảm bảo nhất quán về format và nội dung. Phú tổng hợp và kiểm tra tính nhất quán của toàn bộ báo cáo.
- Báo cáo vai trò cá nhân (Phú): Hoàn thành 10 trang mô tả 3 vai trò: Nhóm trưởng, Developer Race Condition + Admin, QA.

- Seeding dữ liệu demo (Tú): Cập nhật seed.sql với dữ liệu demo thực tế và phong phú: 10 phim các thể loại, 3 rạp tại TP.HCM, suất chiếu cho các ngày trong tuần demo, tài khoản test cho Customer và Admin.
- Demo Script (Phú điều phối): Viết kịch bản demo chi tiết 15 phút:
    1. Demo đăng ký/đăng nhập (2 phút)
    2. Demo xem phim, chọn suất chiếu, bản đồ ghế (3 phút)
    3. Demo Race Condition trực tiếp: 2 người trên 2 máy cùng chọn ghế (3 phút)
    4. Demo thanh toán VNPay (3 phút)
    5. Demo Admin Dashboard (2 phút)
    6. Q&A (2 phút)
- Bug cuối (Phú + Sự): Phát hiện và fix bug minor: Trang "Vé của tôi" không hiển thị tên phim và tên rạp (thiếu JOIN trong SQL). Fix xong trong ngày.
- Performance check (Phú): Đảm bảo server response time < 200ms cho tất cả API thông thường. API có Redis cache < 10ms.

Kết quả buổi họp nhóm 1 (31/03/2026):

- Nội dung: Review Demo Script, thử demo thô (rehearsal lần 1), phân công trình bày.
- Kết quả đạt được:
    - Demo Script hoàn chỉnh và được duyệt.
    - Rehearsal lần 1 thành công, các phần chuyển tiếp mượt.
    - Phân công trình bày: Phú (kiến trúc + Race Condition), Tuấn (Auth + Payment), Tú (Catalog + demo nhanh), Sự (chạy demo UI live).

Kết quả buổi họp nhóm 2 (02/04/2026):

- Nội dung: Rehearsal lần 2, Q&A giả định (mỗi thành viên chuẩn bị 3 câu hỏi khó nhất có thể bị hỏi).
- Câu hỏi khó đã chuẩn bị:
    - *"Nếu Redis bị restart giữa chừng, điều gì xảy ra?"* → DB Layer 2 (FOR UPDATE NOWAIT + Double Check) vẫn bảo vệ hoàn toàn.
    - *"Tại sao dùng NOWAIT thay vì chờ?"* → Fail fast, không block người dùng vô thời hạn.
    - *"SingleFlight khác Mutex như thế nào?"* → Mutex tuần tự từng goroutine, SingleFlight deduplication hoàn toàn.

Mục tiêu tuần sau:

Nộp báo cáo, bảo vệ đồ án.

Tuần 10: 05/04 – 11/04

Công việc thực hiện trong tuần:

- Rà soát lần cuối (Phú + cả nhóm): Đọc lại toàn bộ báo cáo, kiểm tra chính tả, định dạng, tính nhất quán. Đảm bảo tất cả code minh họa trong báo cáo đúng với code thực tế trong dự án.
- Nộp báo cáo (Ngày 07/04/2026): Submit tất cả tài liệu theo yêu cầu của giảng viên.
- Push code lên GitHub (Phú): Cleanup code (xóa file tmp/, comment debug), viết README.md chi tiết hướng dẫn cài đặt và chạy dự án, tag release v1.0.0.
- Chuẩn bị môi trường demo ngày bảo vệ (Sự): Đảm bảo dữ liệu seed sẵn sàng, 2 tài khoản test còn hạn, server khởi động ổn định trên máy demo.
- Rehearsal lần 3 (Ngày 09/04/2026): Chạy demo lần cuối trên máy sẽ trình bày, xác nhận không còn lỗi kỹ thuật, Race Condition demo vẫn hoạt động.
- Bảo vệ đồ án (Ngày 11/04/2026):
    - Trình bày theo Demo Script đã chuẩn bị.
    - Demo live Race Condition: Mở 2 tab, cùng nhấn 1 ghế → chỉ 1 thành công.
    - Q&A với giảng viên.

Kết quả buổi họp nhóm cuối (07/04/2026):

- Nội dung: Đánh giá lại toàn bộ quá trình, nộp báo cáo, confirm lịch bảo vệ.
- Kết quả đạt được:
    - Báo cáo nộp đúng hạn.
    - Code sạch trên GitHub với README đầy đủ.
    - Toàn nhóm sẵn sàng bảo vệ.